15-minute presentation script

Slide 1. Title

Good afternoon, everyone.
Today I will present **VPTQ**, which stands for **Vector Post-Training Quantization for Large Language Models**.

The main question of this paper is very simple:
Can we push LLM weight-only quantization down to around 2 bits, while still keeping usable accuracy and practical inference speed?

The motivation is that large language models are becoming too expensive to deploy, and 2-bit quantization could dramatically reduce memory cost. But at the same time, 2-bit is also an extremely difficult regime, especially for standard scalar quantization. So the paper proposes a vector-quantization-based post-training method called VPTQ.

Slide 2. Outline

Here is the outline of my talk.

First, I will explain the motivation: why 2-bit quantization is attractive, but also difficult.
Second, I will introduce the background: post-training quantization and vector quantization.
Third, I will explain the four main components of VPTQ.
Then I will briefly summarize the full end-to-end algorithm.
Finally, I will show the experimental results.

Slide 3. LLMs Are Too Large to Deploy

Let me start with the motivation.

Modern LLMs are very large. For example, as shown in the paper, storing LLaMA-2 70B in FP16 requires about **140 gigabytes** of memory. That usually means multi-GPU deployment, which is expensive and inconvenient.

A natural solution is **weight-only quantization**, where we compress the weights into low-bit representations after training.
If we go from 16-bit FP16 weights to 2-bit weights, the raw memory for weights can be reduced by about **8 times**.

So the key question is:

Can we push all the way down to 2-bit while still keeping the model useful?

That is the problem this paper tries to solve.

Slide 4. The Limit of Scalar Quantization

The first obstacle is the limitation of **scalar quantization**.

In scalar quantization, each weight is quantized independently.
At 2-bit, each scalar can only take **4 possible values**.

That is often too restrictive.
If the original weights are diverse, mapping each one independently into only four values creates large quantization errors.

This is why methods like GPTQ can work well at 3 or 4 bits, but often collapse at 2 bits.
So the problem is not that scalar quantization is bad in general. The problem is that **2-bit scalar quantization is simply too coarse** for LLM weights.

Slide 5. The Intuition Behind Vector Quantization

This leads to the intuition behind **vector quantization**, or VQ.

Instead of quantizing each weight separately, VQ groups several weights together into a short vector, and then represents that vector by an entry in a **codebook**.

For example, if the vector length is 2 and the codebook has 4 centroids, then instead of storing two scalar values separately, we store one index that points to one of the four learned 2D vectors.

The key advantage is that the codebook can represent **patterns across multiple weights**, not just individual values.
So VQ can exploit correlation and redundancy among nearby weights.

That is why the paper argues that VQ can cover a much richer space than scalar quantization at the same effective bit budget.
For example, with vector length 8 and codebook size 256, one index uses 8 bits to represent 8 weights, which is about **1 effective bit per weight**.

Slide 6. Post-Training Quantization: Deriving the Objective

Now let me move to the background.

The goal of post-training quantization is to replace the original weight matrix (W) with a quantized version (\hat W), while changing the task loss as little as possible.

To formalize this, the paper uses a second-order Taylor expansion of the loss around the original weights.
If the pretrained model is already near a local optimum, the first-order term is close to zero. Then the remaining objective becomes minimizing

[

\Delta W^T H \Delta W,
]

where (H) is the Hessian.

So the Hessian tells us which directions in weight space matter more.
If some direction has large curvature, then quantization error in that direction causes a larger loss increase, so we should quantize it more carefully.

This Hessian-aware perspective is the mathematical starting point of VPTQ.

Slide 7. Computing the Hessian in Practice

Of course, the full Hessian of an LLM is impossible to compute directly.

So in practice, GPTQ-style methods approximate this layer by layer.
For one layer, they use the reconstruction loss between the original layer output and the quantized layer output, based on calibration activations (X).
Then the Hessian becomes proportional to (XX^T), which is much cheaper to compute.

So in practice, the Hessian is not some abstract theoretical object.
It is estimated from layer inputs collected on calibration data, and it gives us a practical way to measure which weight directions are sensitive.

Slide 8. Limitations of Existing VQ Methods

Before introducing VPTQ, the paper compares it to prior VQ-based methods.

The three main baselines here are GPTVQ, AQLM, and QuIP#.

* **GPTVQ** quantizes multiple columns jointly, which leads to error accumulation and limits vector length.
* **AQLM** can achieve strong accuracy, but requires expensive optimization and backpropagation, so quantization is slow.
* **QuIP#** uses Hadamard-based incoherence processing, which helps accuracy, but increases inference-time overhead and hurts throughput.

So the paper’s goal is to improve all three dimensions at once:

* very low bitwidth,
* high accuracy,
* and practical speed.

Slide 9. VPTQ: Overview of Four Components

VPTQ has four main components.

Component 1 is **Channel-Independent Second-Order Optimization**.
Component 2 is **Hessian-Weighted Centroid Initialization**.
Component 3 is **Residual Vector Quantization**.
Component 4 is **Outlier Elimination**.

I will now go through these one by one.

Slide 10. Component 1: Channel-Independent Second-Order Optimization

The first component addresses a core weakness of GPTVQ.

GPTVQ quantizes several columns jointly.
The problem is that errors created inside the current block cannot be corrected immediately.
So quantization errors accumulate inside that block, and as vector length grows, this becomes worse.

VPTQ changes the procedure.
Instead of quantizing several columns at once, it quantizes **one column at a time**.

So it quantizes column A, then immediately propagates the resulting error to the remaining columns.
Then it moves to column B, and does the same thing again.

This column-wise strategy reduces local error accumulation.
That is why the paper calls it **channel-independent second-order optimization**.
This is also what lets VPTQ use longer vectors and larger codebooks more safely than GPTVQ.

Slide 11. Component 1: Why Nearest-Centroid Search Appears

Now let me explain the part that can be confusing at first.

We start from the Hessian-aware objective:

[
\Delta W^T H \Delta W.
]

For a fixed column (q), the paper applies the Lagrange method and derives that the increase in loss is proportional to the reconstruction error of that column:

[
\Delta L_q
==========

\frac{|\hat W_{:,q} - W_{:,q}|*2^2}{2H^{-1}*{qq}}.
]

So for one fixed column, minimizing the second-order loss is equivalent to minimizing how far the quantized column is from the original column.

Then, in VQ, that column is split into short subvectors, and each subvector is represented by a centroid from the codebook.
So the total column error becomes the sum of the Euclidean distances between each subvector and its chosen centroid.

Slide 12. Component 1: The Local Quantization Rule

This gives the local quantization rule.

Once the second-order objective is written in this form, for a fixed column (q), the term (H^{-1}_{qq}) is just a constant.
It does not change which centroid is optimal.

So the local optimization reduces to

[
\arg\min_i |v - C_i|^2.
]

In other words, for the current vector, we simply choose the **nearest centroid in Euclidean distance**.

This is the key simplification:
the overall method is still Hessian-aware, but the local centroid assignment becomes a simple nearest-neighbor search in the codebook.

So VPTQ avoids expensive training-based optimization during this step.

Slide 13. Component 1: Where the Hessian is Still Used

At this point, a natural question is:

If local centroid selection is just Euclidean nearest-neighbor, then where is the Hessian actually used?

The answer is: the Hessian is still essential.

It is used, first, in defining the second-order objective itself.
And second, it is used in **error propagation** after each column is quantized.

After quantizing one column, VPTQ computes the quantization error for that column and propagates it to the remaining columns using the inverse Hessian.
So the centroid lookup itself is simple, but the global error correction remains Hessian-aware.

This is an important point:

VPTQ is not ignoring the Hessian. It is using the Hessian where it matters most, while keeping the local lookup simple.

Slide 14. Component 1: Why This Helps Over GPTVQ

So, compared with GPTVQ, what do we gain?

GPTVQ jointly quantizes multiple columns, which makes internal errors accumulate before they can be corrected.
That makes longer vectors harder to use.

VPTQ instead quantizes one column, corrects the remaining weights immediately, and then moves on.
This reduces accumulated error and works better with longer vectors and larger codebooks.

In short, Component 1 gives VPTQ both **better numerical behavior** and **simpler optimization**.
The paper reports that overall VPTQ uses only about **10.4 to 18.6 percent** of the quantization time of prior state-of-the-art methods, while improving 2-bit accuracy.

Slide 15. Component 2: Hessian-Weighted Centroid Initialization

The second component is **Hessian-weighted centroid initialization**.

K-means depends strongly on the initial centroids.
If the centroids are poorly initialized, quantization quality can be poor.

VPTQ uses a weighted K-means objective, where the weights are the diagonal entries of the Hessian.
So weights that have larger impact on the loss get more importance during centroid initialization.

This makes intuitive sense:
if some region of weight space is more loss-sensitive, then the codebook should represent that region more carefully.

Slide 16. Component 2: Example

This small example makes the idea intuitive.

If we use ordinary K-means, each point contributes equally to the centroid.
But in weighted K-means, points with larger Hessian diagonal values pull the centroid more strongly.

So VPTQ is effectively saying:

**Not all weights are equally important.

Important weights should influence the codebook more.**

That is the role of Component 2.

Slide 17. Component 3: Residual Vector Quantization

The third component is **Residual Vector Quantization**, or RVQ.

The idea is simple.

First, the method quantizes the original vector using one codebook.
Then it looks at the residual error that remains.
Instead of stopping there, it quantizes that residual again using a second codebook.

So the final reconstruction is the sum of the first quantized vector and the quantized residual.

This improves accuracy with only a small additional bit cost.
And the paper highlights that GPTVQ does not support this residual quantization mechanism, which is one of VPTQ's advantages.

Slide 18. Component 4: Outlier Elimination

The fourth component is **Outlier Elimination**.

The problem is that a small fraction of very large weights can dominate the quantization range.
If we use one codebook for everything, the centroids are forced to cover both the normal weights and the outliers.
Then the normal weights, which are the vast majority, may be represented poorly.

VPTQ solves this by separating outliers and giving them their own dedicated codebook.
The main codebook can then focus on the normal weight range.

So this protects the representation quality of the majority of weights, while still handling the outliers explicitly.
The paper typically uses an outlier fraction around 1 percent.

Slide 19. End-to-End Quantization Algorithm

Putting everything together, the per-layer algorithm looks like this.

First, VPTQ optionally separates outliers and quantizes them with an outlier codebook.
Then it performs the main VPTQ procedure with Hessian-weighted centroid initialization and column-wise independent quantization.
Then it can optionally apply residual VQ to compress the remaining error.
Finally, it can do lightweight layer fine-tuning.

At inference time, the quantized model stores indices and codebooks rather than full FP16 weights. Dequantization is mainly codebook lookup, and unlike QuIP#, it does not require Hadamard transforms at inference.
That is one reason the paper reports **1.6 to 1.8 times** higher inference throughput than prior SOTA methods.

Slide 20. Experimental Setup

For experiments, the paper evaluates on LLaMA-2, LLaMA-3, and Mistral models.

The main metrics are WikiText-2 perplexity, C4 perplexity, and average accuracy on five zero-shot QA tasks.
The baselines include GPTQ, GPTVQ, QuIP#, AQLM, and DB-LLM.

For calibration data, the paper follows prior work and uses **128 random segments of the C4 dataset**.

Slide 21. LLaMA-2 7B: 2-bit Results

Now let me show the main result table.

On **LLaMA-2 7B**, VPTQ at about **2.02 bits** achieves WikiText-2 perplexity **6.13**, compared with **6.64** for AQLM at similar bitwidth.
Its throughput is **39.9 tokens per second**, compared with **19.4** for AQLM and **4.4** for QuIP#.
And in quantization time, the paper reports about **2 hours** for VPTQ, compared with **11.07 hours** for one AQLM setting.

So on this table, VPTQ is competitive or better in accuracy, much faster in quantization, and also much faster in throughput.

That is exactly the trade-off the paper wants to win.

Slide 22. LLaMA-3 and Mistral: 2-bit Results

The next slide shows results on harder models.

For **LLaMA-3 8B**, competing 2-bit methods degrade heavily, while VPTQ reaches WikiText-2 perplexity **9.29** and average QA accuracy **60.2**.
For **LLaMA-3 70B**, VPTQ achieves perplexity **5.6** and average QA accuracy **70.9**, clearly better than the shown 2-bit GPTQ baseline.
For **Mistral-7B**, VPTQ at about **2.04 bits** gets WikiText-2 perplexity **5.64**, C4 perplexity **6.43**, and average QA accuracy **63.2**, outperforming QuIP#, AQLM, GPTQ, and GPTVQ in that table.

So the high-level message is:

At 2-bit, many methods collapse, but VPTQ remains relatively stable.

Slide 23. Conclusion

Let me conclude.

VPTQ combines four ideas:

1. channel-independent second-order optimization,
2. Hessian-weighted centroid initialization,
3. residual vector quantization,
4. and outlier elimination.

Together, these let VPTQ operate in the very challenging 2-bit regime while maintaining stronger accuracy than prior baselines.
The paper reports perplexity improvements over prior SOTA, QA gains ranging from about **0.8 percent to 22 percent**, **1.6 to 1.8 times** faster inference throughput, and only **10.4 to 18.6 percent** of the quantization time of prior SOTA methods.

So the final takeaway is:

VPTQ shows that practical 2-bit deployment of LLMs is possible if we combine vector quantization with Hessian-aware second-order optimization and efficient codebook design.

Thank you.